

Chat Transcript

COMP 371 Project 2a (Scala console app) - September 30, 2026

Background lookups such as web searches are omitted.

You

i need help understanding these requirements

Claude

Happy to help. I don't see any requirements attached to your message, though. Could you paste the text here or upload the file?

Once I can see them, let me know which part is confusing (or if it's the whole thing) and I'll break it down in plain terms.

You

(pasted the Project 2a requirements: create the private repo `comp371f26-project2a`, share it with the TA, tutor, and instructor, set up a standard Scala 3 project, use the `consoleapp-java` and `iterators-scala` repos as references, port `Main.java` to Scala, use an LLM to help, and test manually with `sbt stage`, with screenshots for empty input and nonempty input)

Claude

I'll look at the Java app you're meant to port so I can explain it concretely.

Your assignment has two parts: setting up a project, and then writing a small program.

The setup

- Create a private GitHub repo named `comp371f26-project2a`, and add your TA, tutor, and instructor as collaborators so they can see it.
- Use a standard Scala 3 layout: `src/main/scala` for code, a `.gitignore`, a `build.sbt`, `project/plugins.sbt`, and `.github/workflows` for CI. The easiest route is to copy your `shapes-oo-scala` repo and delete the shape classes you don't need.
- Add any libraries you need, such as Apache Commons Collections or a test framework.
- Write Scala 3 with significant indentation (no curly braces where you can avoid them), and copy the compiler options from the course's other Scala examples.

The program

Your job is to port the Java `Main.java` to Scala. I couldn't open that file directly, but I found the README from a sibling version of the same repo, and it describes the behavior:

- The app keeps a running FIFO queue of the last `n` words read from standard input, and after each word it prints the updated queue as `[w1, w2, ..., wn]`, with the newest word last.
- The default capacity is 10, and you can pass a different one as a command-line argument.

- If a line has several words, each is processed separately, with whitespace and other non-word characters as separators.

For example, with capacity 3, typing `w1 w2` prints `[w1]` and then `[w1, w2]`. Once the queue is full, each new word pushes the oldest one out.

The assignment also says to use the `iterators-scala` repo as a reference, in its imperative versions. That means plain `while` loops and mutable state, not functional style yet. Using an LLM to help is explicitly allowed.

What to hand in

- 1 Build with `sbt stage`, which produces a standalone launcher script, and run that script outside of `sbt`.
- 2 Take a screenshot of the empty-input case: start the program and press `Ctrl-D` immediately, which sends end-of-file. It should exit cleanly and print nothing.
- 3 Take a screenshot of a nonempty run similar to the README example above.

The README I found came from a sibling repo, not the exact one you were given, so check your course's version for any differences, such as how the capacity argument is handled.

Do you want to start with the `sbt` project setup or with the Scala code itself?

You

what does this command do? `"yes | ./target/universal/stage/bin/main > /dev/null"`

Claude

It feeds your program an endless stream of input and throws away everything it prints. It's a stress test.

- `yes` is a standard Unix command that prints the letter `y` followed by a newline, forever, as fast as it can.
- `|` pipes that output into the standard input of the next command, so your program reads an endless stream of `y` lines.
- `./target/universal/stage/bin/main` is the launcher script that `sbt stage` generates, so this runs your compiled program outside of `sbt`.
- `> /dev/null` sends your program's standard output to the null device, which discards it. Without this, your terminal would be flooded with `[y, y, y, ...]` lines.

This command never finishes on its own, so you stop it with `Ctrl-C`. The point is to check that your program works on unbounded input, which ties into the "finite vs. infinite/unbounded streams" learning objective in the README.

A correct solution reads one line at a time, processes it, and keeps only the last `n` words in the queue. That way memory use stays flat no matter how long it runs. If your code tried to read all of `stdin` into a list before processing, this command would eventually run out of memory. You can watch for that by running `top` or `htop` in another terminal while the test runs. Memory should stay steady.

You

what libraries should i add to the program?

Claude

For this program you need very little. Copying `shapes-oo-scala` probably already gives you a test library.

Apache Commons Collections (optional). The assignment's mention of "Apache Collections" most likely refers to `CircularFifoQueue`, which the Java version probably uses for the fixed-capacity queue. You can confirm by checking the Java repo's `build.gradle`. In sbt it looks like this:

```
libraryDependencies += "org.apache.commons" % "commons-collections4" % "4.4"
```

The `%` (single) is correct here because it's a Java library, not a Scala one. Check Maven Central for the latest version.

You can also skip the library and use Scala's built-in `scala.collection.mutable.Queue`. You would enqueue each new word and dequeue the oldest when the size exceeds the capacity. That's only a couple of lines, and it avoids a dependency. Since the assignment says the Scala version should replicate the Java one, I'd use Commons Collections unless your instructor says otherwise. Using the same library keeps the two versions closely comparable.

ScalaTest (for testing). The course's other Scala examples use it, so `shapes-oo-scala` likely already has it in `build.sbt`:

```
libraryDependencies += "org.scalatest" %% "scalatest" % "3.2.19" % Test
```

Here `%%` is correct because `ScalaTest` is a Scala library, and `% Test` keeps it out of your final build. The README's learning objectives mention testability, so a few tests are worthwhile even if this part of the project doesn't require them.

Nothing else is needed. The standard library handles reading `stdin` and splitting words, for example with `scala.io.StdIn.readLine()` and a regex split on non-word characters.

You

can you print a transcript for this

Claude

I'm guessing you mean the expected terminal output for your two required scenarios. This is what a correct program should show, based on the README's behavior. I haven't run your code, so your screenshots need to come from your own runs.

Empty input (press Ctrl-D right after starting):

```
$ ./target/universal/stage/bin/main
$
```

The program exits immediately and prints nothing.

Nonempty input with capacity 3 (your typed input is the lines without brackets):

```
$ ./target/universal/stage/bin/main 3
w1 w2
[w1]
[w1, w2]
```

```
w3
[w1, w2, w3]
w4 w5
[w2, w3, w4]
[w3, w4, w5]
w6
[w4, w5, w6]
<press Ctrl-D>
$
```

The README marks the end of input as EOF, which is just a note for Ctrl-D. Your program shouldn't print it.

If you meant a transcript of this conversation instead, tell me and I can put that together.

You

no i meant for our chat

Claude

Here's the transcript of our chat. I left out my background lookups, such as the web searches.

(Claude then printed the conversation so far, which is the same content as this document, so it is not repeated here.)

I can also save this as a file (Markdown or Word) if you'd like.

You

save our chat to pdf